

Composition vs. Inheritance & Design Guidance

Note: Like SOLID, this isn't in your source material — drafted fresh. It's the natural capstone to the OOPS drafts, since it directly resolves tensions raised in the Inheritance, Abstract Classes, and SOLID topics (the Duck problem, the diamond problem, LSP violations) with a single, coherent piece of design guidance.

1. Has-A vs. Is-A — the fundamental distinction

Relationship	Modeled by	Example
Is-a	Inheritance	Dog is an Animal
Has-a	Composition	Car has an Engine

```
// IS-A - inheritance
class Animal { void eat() { /* ... */ } }
class Dog extends Animal { void bark() { /* ... */ } }

// HAS-A - composition
class Engine { void start() { /* ... */ } }
class Car {
    private Engine engine = new Engine(); // Car HAS AN Engine - not "is an" Engine
    void start() { engine.start(); }      // Car DELEGATES to its Engine
}
```

The test that catches most misuse: can you honestly say "X is a Y" and have it hold up under *all* of Y's behavioral guarantees (recall LSP from the SOLID topic)? A Car is **not** an Engine — it *has* one, uses it, and could even be built to swap in a different one. Modeling this as Car extends Engine would be **immediately, obviously wrong** — but the same mistake shows up far more subtly in real designs (the Square/Rectangle case from the SOLID topic is exactly this error, just dressed up in a plausible-sounding "is-a" claim).

2. Why "Favor Composition Over Inheritance" Is Common Advice

Problem 1: The Fragile Base Class Problem

```
class Base {
    void method1() { method2(); }
    void method2() { /* does something */ }
}

class Derived extends Base {
    @Override void method2() { /* overridden differently */ }
}
```

If `Base`'s author later changes **how** `method1()` **internally calls** `method2()` — even without changing either method's public signature — every derived class relying on the old internal calling pattern can silently **break**, despite the derived class's own code never having changed at all. Inheritance exposes a base class's **internal implementation details** to every subclass, not just its public interface — a subclass is coupled to far more than it may realize.

Composition avoids this entirely: `Car` only depends on `Engine`'s **public interface** (`start()`), never on *how* `Engine` internally implements anything — changes inside `Engine` that don't affect its public interface can never break `Car`.

Problem 2: Rigid, Compile-Time-Fixed Hierarchies

Inheritance relationships are fixed **at compile time** — an object's class (and therefore its inherited behavior) can never change after it's created. Composition relationships can be **reassigned at runtime**.

```
class Car {
    private Engine engine;
    Car(Engine engine) { this.engine = engine; }
    void setEngine(Engine engine) { this.engine = engine; } // swap the engine at RUNTIME
    void start() { engine.start(); }
}
```

A `Car`'s engine can be swapped for a different one (say, upgrading from `PetrolEngine` to `ElectricEngine`) **without creating a new Car object at all** — this kind of runtime flexibility is structurally impossible with inheritance, where an object's inherited behavior is locked in at construction and can never change afterward.

Problem 3: Combinatorial Explosion — the direct fix for the Duck problem

Recall the Duck scenario from Abstract Classes & SOLID: `Duck`, `RemoteControlDuck`, `RobotDuck` needed different combinations of `quack()` / `fly()` behavior. What if there were also `SilentDuck` (can fly, can't quack) and `SilentRemoteControlDuck` (can't do either)? With inheritance alone, you'd need a **separate class for every combination** — this grows combinatorially as more behaviors are added.

The composition-based fix (this is literally the textbook Strategy Pattern):

```
interface FlyBehavior { void fly(); }
class CanFly implements FlyBehavior { public void fly() { System.out.println("Flying!"); } }
class CannotFly implements FlyBehavior { public void fly() { System.out.println("Can't fly."); } }

interface QuackBehavior { void quack(); }
class LoudQuack implements QuackBehavior { public void quack() { System.out.println("QUACK!"); } }
class NoQuack implements QuackBehavior { public void quack() { System.out.println("..."); } }

class Duck {
    private FlyBehavior flyBehavior;
    private QuackBehavior quackBehavior;
    Duck(FlyBehavior f, QuackBehavior q) { flyBehavior = f; quackBehavior = q; }
    void performFly() { flyBehavior.fly(); }
    void performQuack() { quackBehavior.quack(); }
}

Duck realDuck = new Duck(new CanFly(), new LoudQuack());
Duck rcDuck = new Duck(new CannotFly(), new NoQuack());
```

No forced fake implementations, no combinatorial class explosion — each `Duck` is *composed* from independently-chosen behavior objects. Adding a new combination (`SilentDuck`) requires **zero new classes** — just `new Duck(new CanFly(), new NoQuack())` . This directly resolves the exact ISP/LSP tension raised earlier, using composition instead of trying to force it through inheritance/interfaces alone.

Interview soundbite: "The Duck problem shows up repeatedly across this whole topic sequence — as an LSP violation, an ISP violation, and now as a combinatorial-explosion problem — because it's genuinely the same underlying design mistake each time: trying to express independent, swappable *behaviors* through a rigid, compile-time inheritance hierarchy. Composition (specifically, the Strategy Pattern) is the standard fix precisely because it lets behaviors be mixed, matched, and even swapped at runtime, instead of needing a new class for every combination."

3. When Inheritance IS the Right Choice

Composition isn't a universal replacement — inheritance remains the right tool when:

- The relationship is a **genuine, stable "is-a"** that holds up under full behavioral substitutability (LSP) — a `Circle` and `Square` both genuinely *are* `Shape` s, with no hidden contract violations.
- You want to **share a substantial, stable amount of common implementation** across closely related types, and that shared implementation isn't likely to need per-instance runtime swapping.
- The hierarchy is genuinely **shallow and unlikely to grow combinatorially** — a small, stable family of related types, not one where new independent dimensions of variation keep appearing.

Interview soundbite: "'Favor composition over inheritance' doesn't mean 'never use inheritance' — it means: reach for inheritance only when the is-a relationship is genuinely solid and stable, and reach for composition by default whenever you're modeling capabilities/behaviors that might vary independently, need to change at runtime, or don't cleanly satisfy LSP."

4. Aggregation vs. Composition — a lifecycle distinction (a common follow-up question)

Both are "has-a" relationships, but they differ in **ownership and lifecycle**:

	Composition (strong "has-a")	Aggregation (weak "has-a")
Lifecycle	The part cannot exist independently of the whole — created and destroyed with it	The part can exist independently — it may outlive the whole, or be shared by several wholes
Example	<code>House</code> and <code>Room</code> — a <code>Room</code> doesn't meaningfully exist without its <code>House</code>	<code>University</code> and <code>Professor</code> — a <code>Professor</code> exists independently, and could work at a different university, or none at all

```

class House {
    private final Room room = new Room(); // COMPOSITION: House creates & owns Room's lifecycle
}
class University {
    private List<Professor> professors; // AGGREGATION: Professors are passed in / exist independently
    University(List<Professor> professors) { this.profs = professors; }
}

```

Interview Questions With Answers

Q1. What's the precise test for whether a relationship should be modeled with inheritance ("is-a") or composition ("has-a")?

Answer: The test is whether the relationship holds up under full behavioral substitutability — can you honestly claim "every instance of the subtype can be used anywhere an instance of the supertype is expected, without breaking correctness" (the Liskov Substitution Principle)? If the relationship is really about one object *using* or *containing* another, rather than genuinely *being* a specialized version of it that fully honors all of its behavioral guarantees, it should be modeled as composition (has-a) rather than inheritance (is-a) — a Car using an Engine is a clear has-a; forcing it into "Car extends Engine" would immediately fail this test.

Q2. What is the fragile base class problem, and why does composition avoid it?

Answer: The fragile base class problem occurs because inheritance couples a subclass not just to a base class's public interface, but to internal implementation details of how the base class's methods interact with each other — a base class author changing internal calling patterns (even without changing any public signatures) can silently break derived classes relying on the old internal behavior, even though those derived classes' own code never changed. Composition avoids this because a composing class only ever depends on the composed object's public interface (calling its public methods) — it has no visibility into or dependency on how that object's methods are internally implemented or how they call each other, so internal changes that preserve the public interface can never break the composing class.

Q3. Why is inheritance described as "fixed at compile time" while composition can be "reassigned at runtime," and why does this matter practically?

Answer: An object's class (and therefore everything it inherits) is determined at the moment it's constructed and can never change for the lifetime of that object — you cannot make an already-created `Dog` object become a `Cat` at runtime. A composed relationship, by contrast, is just a reference to another object stored in a field — that reference can be reassigned at any point during the composing object's lifetime (e.g., `car.setEngine(newEngine)`), changing the composing object's effective behavior without creating a new object at all. This matters practically whenever a system needs to change an object's behavior dynamically at runtime (e.g., swapping strategies, upgrading a component) — something inheritance structurally cannot support, since it commits to a fixed type at construction.

Q4. How does the composition-based Strategy Pattern solve the "combinatorial explosion" problem that a pure-inheritance approach to the Duck example runs into?

Answer: With pure inheritance, every independent combination of behaviors (can-fly/can't-fly × can-quack/can't-quack, and any further behaviors added later) would require its own dedicated subclass, and the number of needed subclasses grows multiplicatively as more independent behavior dimensions are added. The Strategy Pattern instead extracts each behavior (flying, quacking) into its own small interface with interchangeable implementations, and has the main class (`Duck`) hold references to whichever specific behavior objects it needs, supplied via its constructor. This means every combination is achieved just by composing existing behavior objects together — no new class is needed for a new combination, only (at most) a new behavior implementation if a genuinely new behavior variant is needed, which grows additively rather than multiplicatively as the design evolves.

Q5. Give an example of when inheritance is genuinely the right choice, and explain why composition wouldn't be a clear improvement there.

Answer: A family of `Shape` types (`Circle`, `Square`, `Triangle`) all genuinely, stably "are" shapes, fully satisfying the behavioral contract of a shared `Shape` interface (e.g., all correctly support `area()` with no hidden violations) — this is a solid, stable is-a relationship with no risk of the LSP violations or combinatorial growth that motivate composition elsewhere. Modeling this with composition instead (e.g., a `Shape` class holding a "shape behavior" object) would add indirection without solving any actual problem, since there's no need for runtime-swappable shape-ness, no fragile-base-class risk from meaningfully differing internal implementations, and no combinatorial explosion of independent behavior dimensions — inheritance here is simpler and just as correct.

Q6. What's the difference between composition and aggregation, given both are "has-a" relationships?

Answer: The difference is about lifecycle and ownership. In composition (a "strong" has-a), the contained part cannot meaningfully exist independently of the whole — it's created and destroyed along with it (e.g., a `Room` doesn't exist independently of its `House` — it's specifically *that house's* room). In aggregation (a "weak" has-a), the contained part has an independent existence and lifecycle — it can exist before, after, or entirely separately from the whole, and might even be shared across multiple

wholes (e.g., a `Professor` exists independently of any specific `University` and could move to a different one, or exist without being currently employed by any university at all).

Q7. Why does the text describe the Duck problem as "the same underlying design mistake" appearing as an LSP violation, an ISP violation, and a combinatorial explosion problem, across three different topics?

Answer: All three framings are different symptoms of the same root cause: trying to represent independent, potentially-varying *behaviors* (flying, quacking) as if they were fixed, structural properties of a rigid type hierarchy. Framed through LSP, forcing `RemoteControlDuck` to inherit a `fly()` method it can't honestly fulfill breaks substitutability. Framed through ISP, a `Duck` interface bundling both `fly()` and `quack()` forces implementers that only support one capability to fake the other. Framed as combinatorial explosion, needing a distinct subclass for every independent combination of behaviors an inheritance hierarchy would require grows unmanageably as more behavior dimensions are added. Composition (specifically the Strategy Pattern) resolves all three simultaneously because it stops trying to encode independently-varying behavior as fixed type structure in the first place — it treats behaviors as separately swappable objects instead, which is the actual root fix underlying all three symptoms.

Q8. Scenario: A media player application starts with a `Player` base class, then grows `AudioPlayer` and `VideoPlayer` subclasses. Product requirements later demand that playback can support multiple simultaneous features independently — variable playback speed (normal/slow/fast), and optional subtitles (on/off) — and any player type should be able to combine any speed setting with any subtitle setting. A developer proposes creating `SlowAudioPlayerWithSubtitles`, `FastAudioPlayerNoSubtitles`, `SlowVideoPlayerWithSubtitles`, and so on, as needed. Evaluate this proposal and recommend a better design using concepts from this topic.

Answer: This proposal recreates the exact combinatorial explosion problem the Duck example illustrates — with 2 player types × 3 speed settings × 2 subtitle settings, that's already 12 potential subclasses, and adding any new independent dimension (e.g., a third media type, or a new feature like audio-description tracks) would multiply the required subclass count further, quickly becoming unmanageable. The better design extracts "playback speed" and "subtitle display" as independent, composable behaviors, following the same Strategy Pattern approach used for the Duck problem: a `PlaybackSpeedStrategy` interface (with `Normal` / `Slow` / `Fast` implementations) and a `SubtitleStrategy` interface (with `On` / `Off` implementations), each composed into a `Player` (or its `AudioPlayer` / `VideoPlayer` subclasses, if that base distinction is itself a stable, genuine is-a relationship worth keeping) via constructor injection or setters. This reduces the design to needing only as many classes as there are actual distinct behavior variants (a handful), with every combination achieved by composing them together at runtime — new speed or subtitle options can be added as single new small classes, without any multiplicative growth in the number of player classes needed.